ChatGPT Iniciar sesión Registrarse gratuitamente

en java split "6, 7, 1" y colocar en tres variables o arreglo enteros

En Java puedes dividir el string `"6, 7, 1"` usando `split()` y luego convertir cada parte a entero. Aquí tienes varias formas:

✅ Opción 1: Usando arreglo de enteros

Java

```java
String texto = "6, 7, 1";

// Separar por coma (y opcionalmente espacio)
String[] partes = texto.split(",\\s*");

// Crear arreglo de enteros
int[] numeros = new int[partes.length];

for (int i = 0; i < partes.length; i++) {
    numeros[i] = Integer.parseInt(partes[i]);
}

// Resultado: numeros = [6, 7, 1]
```

✅ Opción 2: Guardar en variables individuales

Java

```java
String texto = "6, 7, 1";

String[] partes = texto.split(",\\s*");

int a = Integer.parseInt(partes[0]);
int b = Integer.parseInt(partes[1]);
int c = Integer.parseInt(partes[2]);
```

✅ Opción 3: Usando streams (más moderno)

```java
String texto = "6, 7, 1";

int[] numeros = java.util.Arrays.stream(texto.split(",\\s*"))
                                .mapToInt(Integer::parseInt)
                                .toArray();
```

💡 Nota: `",\\s*"` permite manejar espacios opcionales después de la coma, así funciona con `"6,7,1"` o `"6, 7, 1"`.

Si quieres, te muestro cómo validar errores (por ejemplo si el string viene mal formado).

Comparar con Claude

regex en java para un texto que DEBE incluir 3 valores enteros separados por comas, como por ejemplo 3, 5, 9. el primero y el segundo deben estar entre 0 y 8 y el tercero entre 1 y 9

Puedes usar esta expresión regular en Java:

```java
String regex = "^[0-8],\\s*[0-8],\\s*[1-9]$";
```

🔎 Explicación:

- `^` → inicio del texto
- `[0-8]` → primer número entre 0 y 8
- `,\\s*` → coma seguida de espacios opcionales
- `[0-8]` → segundo número entre 0 y 8
- `,\\s*` → otra coma con espacios opcionales
- `[1-9]` → tercer número entre 1 y 9

- `$` → fin del texto

✅ Ejemplo en Java:

Java

```java
String texto = "3, 5, 9";

if (texto.matches("^[0-8],\\s*[0-8],\\s*[1-9]$")) {
    System.out.println("Formato válido");
} else {
    System.out.println("Formato inválido");
}
```

✔️ Casos válidos:

- `3,5,9`
- `0, 8, 1`
- `7,2,9`

❌ Casos inválidos:

- `9,5,3` (primer número fuera de rango)
- `3,9,1` (segundo fuera de rango)
- `3,5,0` (tercero fuera de rango)
- `3,5` (faltan valores)

Si quisieras permitir números de más de un dígito (por ejemplo 10, 12, etc.), habría que ajustar la regex.

Comparar con Claude